

DAY - 57

1. kth largest element in Array - min heap.

maintain a min heap of size k

keep only k largest elements. TC - $(N \log k)$

The root = answer.

SC - $O(k)$.

```
def findK(nums, k):
```

```
    minheap = []
```

```
    for num in nums:
```

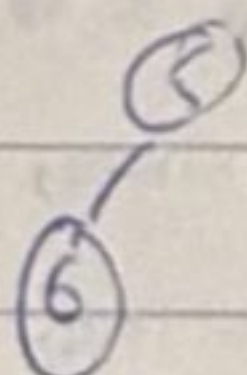
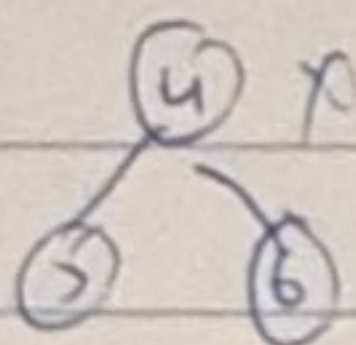
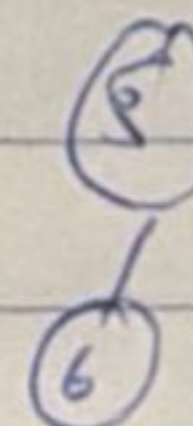
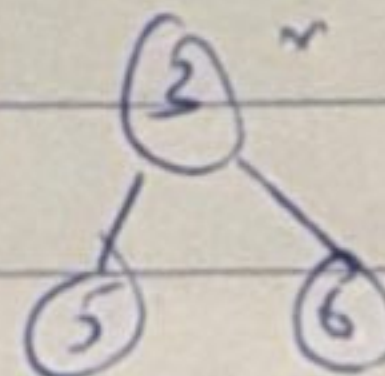
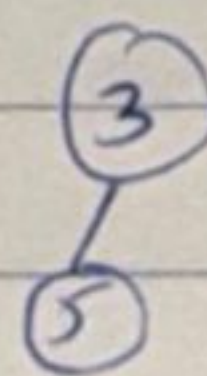
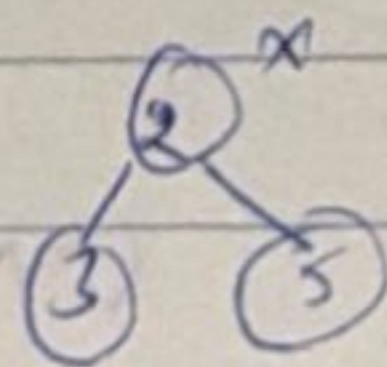
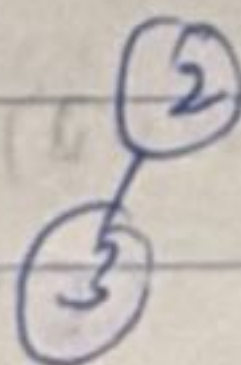
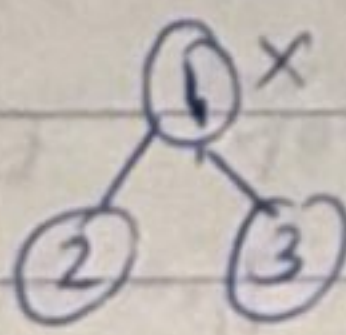
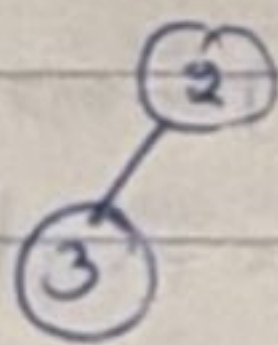
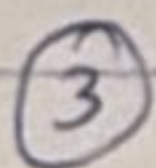
```
        heapq.heappush(minheap, num)
```

```
        if len(minheap) > k:
```

```
            heapq.heappop(minheap)
```

```
    return minheap[0]
```

nums = [3, 2, 1, 5, 6, 4] k = 2.



so at any point k elements in heap; kth largest in root.

2. kth largest element in a stream:

--init--() - runs only once when object is created
add() - doesn't call init method.

```
def __init__(self, k, nums):  
    self.k = k  
    self.minheap = []  
    for num in nums:  
        heapq.heappush(self.minheap, num)  
        if len(self.minheap) > k:  
            heapq.heappop(self.minheap)  
  
def add(self, val):  
    heapq.heappush(self.minheap, val)  
    if len(self.minheap) > self.k:  
        heapq.heappop(self.minheap)  
    return self.minheap[0]
```

3. Last Stone Weight:

Take 2 heaviest stones
smash them;

if equal $\rightarrow$ both delete

if different $\rightarrow$ push back difference.

Note: Use Max Heap

but python has only MinHeap so store -ve values

def lastStoneWeight(stones):

maxheap = [-s for s in stones]

heapq.heapify(maxheap)

while len(maxheap) > 1:

first = -heapq.heappop(maxheap)

second = -heapq.heappop(maxheap)

if first != second:

heapq.heappush(maxheap, -(first - second))

return -maxheap[0] if maxheap else 0

4. kth closest point to origin:

distance = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ since one point is (0,0)

distance = $\sqrt{x^2 + y^2}$

we can use $x^2 + y^2$

Approach:

- Maintain heap of size k

- Remove farthest point i.e., max distance

def kClosest(points, k):

maxheap = []

for x, y in points:

dist = x*x + y*y

heapq.heappush(maxheap, (-dist, x, y))

if len(maxheap) > k:

heapq.heappop(maxheap)

return [(x, y) for (-dist, x, y) in maxheap]